



Lista de Exercícios — Dependabilidade e Teste de Software

Prof. Iaçanã Ianiski Weber

Parte 1 — Conceitos de Dependabilidade

Exercício 1. Defina, com suas palavras, os conceitos de *fault* (defeito), *error* (erro de estado) e *failure* (falha de serviço). Em seguida, dê um exemplo concreto de software em que um *fault* está presente no código, mas nenhum *error* se manifesta durante a execução normal.

Exercício 2. Considere o seguinte cenário: um microsserviço de pagamentos processa uma requisição de débito, mas um bug faz com que o saldo seja decrementado duas vezes quando ocorre um timeout seguido de retry pelo cliente.

- a) Identifique o *fault*, o *error* e a *failure* neste cenário.
- b) A qual classe de semântica de falha (crash, omission, timing, value, Byzantine) esse comportamento pertence? Justifique.
- c) Proponha um mecanismo para evitar o problema (idempotência, deduplicação ou outro).

Exercício 3. Classifique cada situação abaixo na semântica de falha mais adequada (crash, omission, timing, value/response, Byzantine):

- a) Um servidor web fecha a conexão e reinicia após uma exceção não tratada.
- b) Um sensor de temperatura retorna sempre o valor 25°C, mesmo quando a temperatura real é 80°C.
- c) Um serviço responde corretamente, mas após 12 segundos quando o SLA exige resposta em até 2 segundos.
- d) Um nó de consenso envia o valor “A” para um peer e o valor “B” para outro peer na mesma rodada.
- e) Uma mensagem de heartbeat é enviada pelo servidor, mas se perde na rede e nunca chega ao monitor.

Exercício 4. Explique a diferença entre *fail-silent*, *fail-stop* e *fail-fast*. Para cada um, dê um exemplo de sistema ou componente que se beneficia desse comportamento.

Exercício 5. Um sistema de e-commerce possui a seguinte especificação: “cada pedido deve ser processado exatamente uma vez”.

- a) Explique por que a semântica *exactly-once* é difícil de garantir em um sistema distribuído.
- b) Compare as semânticas *at-least-once* e *at-most-once*, indicando vantagens e riscos de cada uma nesse contexto.
- c) Descreva como a combinação de idempotência + chave de deduplicação pode aproximar o comportamento de *exactly-once*.

Exercício 6. Considere um sistema IoT com os seguintes componentes: sensores, gateway local, rede (Wi-Fi), serviço na nuvem e dashboard de monitoramento.

- Identifique pelo menos 3 fronteiras de contenção (*fault containment regions*) nesse sistema.
- Para cada fronteira, descreva: que tipo de erro ela consegue conter e o que ela **não** consegue conter.
- Proponha uma estratégia de detecção para cada fronteira (check semântico, timeout, heartbeat, etc.).

Exercício 7. Um serviço A depende de um serviço B. Quando B fica lento, A faz retries agressivos (sem backoff), o que sobrecarrega B e causa uma cascata de falhas.

- Explique o fenômeno de *retry storm* e por que ele amplifica falhas.
- Proponha pelo menos 3 mecanismos para mitigar esse problema.
- Qual é o papel do *circuit breaker* nesse cenário? Descreva seu funcionamento básico.

Exercício 8. Compare as quatro estratégias de recuperação abaixo. Para cada uma, dê um cenário em que ela é a mais adequada:

- Restart (reinicialização do processo)
- Rollback (checkpoint + retorno ao estado anterior)
- Transação (atomicidade garantida por commit/abort)
- Compensação (saga: desfazer efeitos parciais em sistemas distribuídos)

Parte 2 — Confiabilidade e Disponibilidade

Exercício 9. Um servidor operou continuamente por 60 dias. Nesse período, ocorreram 4 falhas. Os tempos de reparo foram: 2,0h; 1,0h; 3,0h; 0,5h.

- Calcule o MTBF (em horas).
- Calcule o MTTR.
- Calcule a disponibilidade A.
- Estime o *downtime* anual desse sistema.

Exercício 10. Considere dois sistemas reparáveis:

- Sistema P: MTBF = 800 h, MTTR = 16 h
- Sistema Q: MTBF = 200 h, MTTR = 1 h

- Calcule a disponibilidade de cada sistema.
- Qual sistema é mais **confiável**? E qual é mais **disponível**?
- Explique, em termos práticos, por que confiabilidade e disponibilidade podem apontar para sistemas diferentes.

Exercício 11. Assumindo o modelo exponencial de falhas com MTBF = 1000 h:

- Qual a taxa de falha λ ?

- b) Calcule $R(200)$, ou seja, a probabilidade de operar sem falha por 200 horas.
- c) Calcule $R(1000)$.
- d) Se a missão exige confiabilidade de pelo menos 95% em 100 horas, esse sistema atende?

Exercício 12. Uma empresa deseja garantir um SLA de $A \geq 99,9\%$ para sua plataforma. O sistema atual tem $MTBF = 500$ h.

- a) Qual deve ser o MTTR máximo para atender esse SLA?
- b) Com esse MTTR máximo, estime o *downtime* anual permitido.
- c) Se na prática o $MTTR = 2$ h, o SLA é atendido? Justifique.

Exercício 13. Diferencie MTTF e MTBF. Em que tipo de sistema cada métrica é mais apropriada? Dê um exemplo para cada.

Exercício 14. Um sistema deve operar continuamente por 48 horas para uma missão crítica. Duas opções estão disponíveis:

- Opção A: $MTBF = 400$ h
- Opção B: $MTBF = 1200$ h

- a) Calcule $R(48)$ para cada opção (modelo exponencial).
- b) Qual o ganho em pontos percentuais de confiabilidade da opção B sobre a opção A?
- c) Se o custo da opção B é $3\times$ maior, você recomendaria a troca? Justifique considerando o contexto de missão crítica.

Parte 3 — Safety, Security e Resiliência

Exercício 15. Defina *safety* e *security* no contexto de dependabilidade. Qual a principal diferença em termos de origem da ameaça (acidental vs. intencional)?

Exercício 16. Considere um sistema de controle de tráfego ferroviário. Um atacante compromete a rede de comunicação e injeta comandos falsos de sinalização.

- a) Esse cenário é primariamente um problema de *safety* ou de *security*? Justifique.
- b) Explique como uma vulnerabilidade de *security* pode se tornar um risco de *safety*.
- c) Proponha pelo menos duas mitigações que enderecem tanto o aspecto de *safety* quanto o de *security*.

Exercício 17. Utilizando o modelo STRIDE, identifique pelo menos uma ameaça de cada categoria para um sistema de atualização de firmware OTA (*Over-The-Air*) de dispositivos IoT. Preencha a tabela:

Categoria STRIDE	Ameaça identificada	Propriedade violada
Spoofting		
Tampering		
Repudiation		
Information Disclosure		
Denial of Service		
Elevation of Privilege		

Exercício 18. Explique o conceito de *hazard* e diferencie-o de *failure*. Em seguida, para um sistema de piloto automático de drone de entrega, identifique:

- Dois possíveis *hazards*.
- Para cada hazard, uma *safety constraint* (restrição de segurança funcional).
- Uma mitigação arquitetural para cada hazard.

Exercício 19. Considere o seguinte trecho de uma FMEA para um sistema de monitoramento de pacientes em UTI:

Item	Modo de falha	Efeito	Mitigação
Sensor SpO2	Leitura travada (<i>stuck-at</i>)	?	?
Rede Wi-Fi	Perda de pacotes	?	?
Software de alerta	Alarme não disparado	?	?

Complete a tabela, preenchendo os efeitos e as mitigações para cada modo de falha.

Exercício 20. Desenhe uma *Fault Tree* (árvore de falhas) simplificada para o evento topo: “Paciente não recebe alerta de condição crítica”. Considere pelo menos as seguintes causas:

- Sensor com leitura incorreta
- Rede indisponível
- Software de alerta com bug
- Dispositivo de notificação sem bateria

Indique quais combinações usam portas AND e quais usam portas OR. Justifique suas escolhas.

Exercício 21. Explique o que são as propriedades CIA (Confidentiality, Integrity, Availability) e AAA (Authentication, Authorization, Accountability). Para cada propriedade, dê um exemplo de violação em um sistema de prontuário eletrônico hospitalar.

Exercício 22. Defina *resiliência* no contexto de sistemas computacionais. Em seguida, para cada uma das quatro capacidades (antecipar, resistir, recuperar, adaptar), dê um exemplo concreto de mecanismo ou prática em um sistema web de alta disponibilidade.

Exercício 23. Explique por que projetar safety e security separadamente pode ser arriscado. Dê um exemplo concreto em que uma mitigação de security cria um novo risco de safety (ou vice-versa).

Parte 4 — Teste de Software: Caixa Preta

Exercício 24. Uma função recebe um inteiro *idade* e retorna uma string conforme as regras:

- Se $idade < 0$ ou $idade > 150$: retorna “inválido”
- Se $0 \leq idade \leq 12$: retorna “criança”
- Se $13 \leq idade \leq 17$: retorna “adolescente”
- Se $18 \leq idade \leq 64$: retorna “adulto”
- Se $65 \leq idade \leq 150$: retorna “idoso”

- Identifique todas as classes de equivalência (válidas e inválidas).
- Projete um caso de teste representativo para cada classe.
- Identifique todas as fronteiras e projete casos de teste para os valores *on* e *off* de cada fronteira.

Exercício 25. Considere a seguinte especificação de uma função que calcula o desconto em uma loja online:

```
float calcular_desconto(float valor, int tipo_cliente, int cupom);
```

Parâmetro	Tipo	Domínio	Descrição
valor	float	(0,10000]	Valor da compra em reais
tipo_cliente	int	{1,2,3}	1=normal, 2=premium, 3=VIP
cupom	int	{0,1}	0=sem cupom, 1=com cupom

Regras:

- Se qualquer parâmetro estiver fora do domínio, retornar -1 .
- Desconto base: normal = 0%, premium = 10%, VIP = 20%.
- Se `cupom = 1`, adicionar 5% ao desconto.
- Desconto máximo: 25%. Se o desconto calculado ultrapassar 25%, limitar a 25%.
- Retorno: valor do desconto em reais ($\text{valor} \times \text{percentual}$).

- a) Identifique todas as classes de equivalência inválidas.
- b) Identifique todas as classes de equivalência válidas (considere as combinações de tipo de cliente e cupom).
- c) Identifique as fronteiras relevantes e projete casos de teste para valores *on* e *off*.
- d) Monte uma suíte de testes consolidada em tabela com colunas: ID, valor, tipo_cliente, cupom, resultado esperado, técnica, classe coberta.

Exercício 26. Para a especificação do Exercício 25, identifique pelo menos 3 defeitos plausíveis que um programador poderia introduzir na implementação. Para cada defeito, indique qual caso de teste da sua suíte o detectaria.

Exercício 27. Uma função recebe três inteiros representando os lados de um triângulo e classifica-o como equilátero, isósceles, escaleno ou erro (não forma triângulo). Sabe-se que a implementação contém o seguinte defeito: a condição de desigualdade triangular usa $\geq$ em vez de $>$ (ou seja, $a+b \geq c$ em vez de $a+b > c$).

- a) Qual caso de teste detecta esse defeito? Forneça valores concretos.
- b) Explique por que um caso de teste com valores “genéricos” (ex.: $a = 3, b = 4, c = 5$) **não** detectaria esse defeito.
- c) Qual técnica de teste (particionamento ou valor limite) é mais eficaz para encontrar esse tipo de bug?

Parte 5 — Teste de Software: Caixa Branca

Exercício 28. Considere o seguinte código em C:

```
1 int verificar_acesso(int nivel, int autenticado) {
2     int resultado = 0;
3     if (nivel >= 3 && autenticado == 1) {
4         resultado = 1;
5     }
6     if (nivel >= 5) {
7         resultado = 2;
8     }
9     return resultado;
10 }
```

- Desenhe o grafo de fluxo de controle (CFG) deste código.
- Quantos caminhos independentes existem? Justifique usando a complexidade ciclomática.
- Projete um conjunto mínimo de casos de teste que alcance **cobertura de ramificações** (branch coverage) de 100%.
- Os mesmos casos de teste garantem cobertura de comandos (statement coverage) de 100%? Justifique.

Exercício 29. Considere o seguinte código:

```

1  int calcular_bonus(int vendas, int tempo_empresa) {
2      int bonus = 0;
3      if (vendas > 100 || tempo_empresa > 5) {
4          bonus = 500;
5          if (vendas > 200 && tempo_empresa > 10) {
6              bonus = 1500;
7          }
8      }
9      return bonus;
10 }
```

- Desenhe o grafo de fluxo de controle (CFG).
- Calcule a complexidade ciclomática.
- Projete casos de teste para alcançar 100% de cobertura de ramificações (branch coverage).
- É possível alcançar 100% de cobertura de caminhos com menos de 4 casos de teste? Justifique.

Exercício 30. Para o código abaixo, projete um conjunto de casos de teste que atinja 100% de cobertura de condição/decisão (cada condição individual avaliada como verdadeira e falsa pelo menos uma vez):

```

1  int classificar(int x, int y) {
2      if (x > 0 && y > 0) {
3          return 1; // primeiro quadrante
4      } else if (x < 0 || y < 0) {
5          return -1; // fora do primeiro quadrante
6      }
7      return 0; // sobre um eixo
8  }
```

- Liste todas as condições atômicas presentes no código.
- Para cada condição atômica, indique em quais casos de teste ela é verdadeira e em quais é falsa.
- Quantos casos de teste são necessários no mínimo? Apresente-os com os valores de x, y e o retorno esperado.

Exercício 31. Considere o seguinte código:

```

1 int processar(int a, int b) {
2     int r = a;
3     if (a > b) {
4         r = a - b;
5     } else {
6         r = b - a;
7     }
8     if (r == 0) {
9         r = -1;
10    }
11    return r;
12 }

```

- Identifique todos os caminhos possíveis no código.
- Projete casos de teste para cobertura de caminhos (path coverage) de 100%.
- Existe algum caminho infactível? Se sim, qual e por quê?

Exercício 32. Explique a hierarquia de critérios de cobertura: cobertura de comandos (statement) $\subseteq$ cobertura de ramificações (branch) $\subseteq$ cobertura de caminhos (path). Para cada par consecutivo, dê um exemplo de código em que alcançar o critério mais fraco **não** garante o mais forte.

Parte 6 — Integração: Dependabilidade e Teste

Exercício 33. Considere uma função que implementa um mecanismo de retry com backoff exponencial:

```

1 int retry_request(int max_retries) {
2     int tentativas = 0;
3     int sucesso = 0;
4     while (tentativas < max_retries && sucesso == 0) {
5         sucesso = enviar_requisicao();
6         if (sucesso == 0) {
7             tentativas++;
8             esperar(tentativas * 2); // backoff
9         }
10    }
11    return sucesso;
12 }

```

- Projete casos de teste de caixa branca para cobrir todas as ramificações deste código.
- Projete casos de teste de caixa preta considerando as classes de equivalência: sucesso na primeira tentativa, sucesso após retries, falha após esgotar tentativas.
- Este código implementa adequadamente backoff exponencial? Se não, identifique o problema.
- Do ponto de vista de dependabilidade, que mecanismos adicionais poderiam ser incorporados (jitter, circuit breaker, etc.)?

Exercício 34. Um sistema tem $MTBF = 720$ h e um conjunto de testes automatizados que roda a cada deploy. Após uma melhoria no processo de testes (adição de testes de valor limite e cobertura de ramificações), o MTBF sobe para 1200 h, mantendo $MTTR = 2$ h.

- a) Calcule a disponibilidade antes e depois da melhoria.
- b) Calcule $R(48)$ (modelo exponencial) antes e depois.
- c) De que forma teste de software contribui para **prevenção de falhas** (remoção de *faults*)?
- d) Qual a relação entre cobertura de testes e confiabilidade? Um sistema com 100% de cobertura de ramificações é necessariamente confiável?

Exercício 35. Considere a seguinte especificação de um mecanismo de idempotência:

```

1 int processar_pedido(int pedido_id, int valor) {
2     if (pedido_id <= 0 || valor <= 0) {
3         return -1; // entrada invalida
4     }
5     if (ja_processado(pedido_id)) {
6         return buscar_resultado(pedido_id); // replay
7     }
8     int resultado = debitar(valor);
9     registrar(pedido_id, resultado);
10    return resultado;
11 }

```

- a) Identifique as classes de equivalência (válidas e inválidas) para o teste de caixa preta desta função.
- b) Projete casos de teste de caixa branca para 100% de cobertura de ramificações.
- c) Explique como esse mecanismo de idempotência se relaciona com a dependabilidade do sistema (especificamente com a cadeia *fault* → *error* → *failure*).
- d) O que acontece se a função **registrar** falhar entre **debitar** e **registrar**? Qual estratégia de recuperação resolveria esse problema?

Exercício 36. Relacione as colunas, associando cada conceito de dependabilidade ao papel que o teste de software desempenha:

Conceito	Papel do Teste
(1) Remoção de faults	(A) Teste de stress/carga para verificar limites
(2) Detecção de erros	(B) Teste funcional para encontrar e corrigir bugs antes do deploy
(3) Validação de safety constraints	(C) Teste de regressão para garantir que correções não introduzem novos defeitos
(4) Verificação de tolerância a falhas	(D) Teste de injeção de falhas para verificar mecanismos de recuperação
(5) Manutenção da confiabilidade	(E) Teste contra requisitos de segurança funcional derivados de hazard analysis

Exercício 37. Um engenheiro afirma: “Se todos os testes passam, o sistema é confiável”. Critique essa afirmação considerando:

- a) A distinção entre verificação e validação.
- b) As limitações de cobertura de testes (o que 100% de branch coverage **não** garante).
- c) A diferença entre ausência de falhas observadas e ausência de defeitos.
- d) O papel de falhas operacionais (configuração, ambiente, carga) que testes unitários não cobrem.